



Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики і програмної інженерії

Звіт

з дисципліни «Технології розподілених обчислень»

Комп'ютерний практикум №5

«Застосування високорівневих засобів паралельного програмування
для побудови алгоритмів імітації та дослідження їх ефективності»

Виконав:

Студент III курсу

гр. ІІІ-33

Соколов О. В.

Прийняв:

Дифучин А. Ю.

“11” квітня 2026 р.

КОМП'ЮТЕРНИЙ ПРАКТИКУМ №5

Тема: Застосування високорівневих засобів паралельного програмування для побудови алгоритмів імітації та дослідження їх ефективності

Завдання:

1. З використанням пулу потоків побудувати алгоритм імітації багатоканальної системи масового обслуговування з обмеженою чергою, відтворюючи функціонування кожного каналу обслуговування в окремій підзадачі. Результатом виконання алгоритму є розраховані значення середньої довжини черги та ймовірності відмови. 40 балів.

2. З використанням багатопоточної технології організувати паралельне виконання прогонів імітаційної моделі СМО для отримання статистично значимої оцінки середньої довжини черги та ймовірності відмови. 20 балів.

3. Виводити результати імітаційного моделювання (стан моделі та чисельні значення вихідних змінних) в окремому потоці для динамічного відтворення імітації системи. 20 балів.

4. Побудувати теоретичні оцінки показників ефективності для одного з алгоритмів практичних завдань 2-5. 20 балів.

ВИКОНАННЯ

Завдання 1

Змодельовано N-канальну СМО з обмеженим буфером. Інтервали між надходженнями заявок – випадкові з рівномірним розподілом на інтервалі, симетричному відносно заданого середнього; час обслуговування – випадковий за нормальним законом (з обрізанням до додатних значень). Якщо є вільний канал, заявка обслуговується негайно; інакше вона стає в чергу; при переповненні буфера заявка отримує відмову. Кожен канал обслуговування відповідає окремій підзадачі у складі пулу потоків; окремий потік моделює надходження заявок, ще один через рівні інтервали фіксує довжину черги очікування для подальшого усереднення. Ймовірність відмови визначається як відношення числа відмов до числа спроб входу в систему. Умова зупинки імітації: обслуговано не менш як 1000 заявок.

Схема відповідає ідеї виробник – споживачі: потік надходження постачає заявки в обмежений буфер; робочі потоки каналів «споживають» заявки з буфера, утримуючи їх на час обслуговування; при неможливості зайняти місце в буфері заявка відкидається.

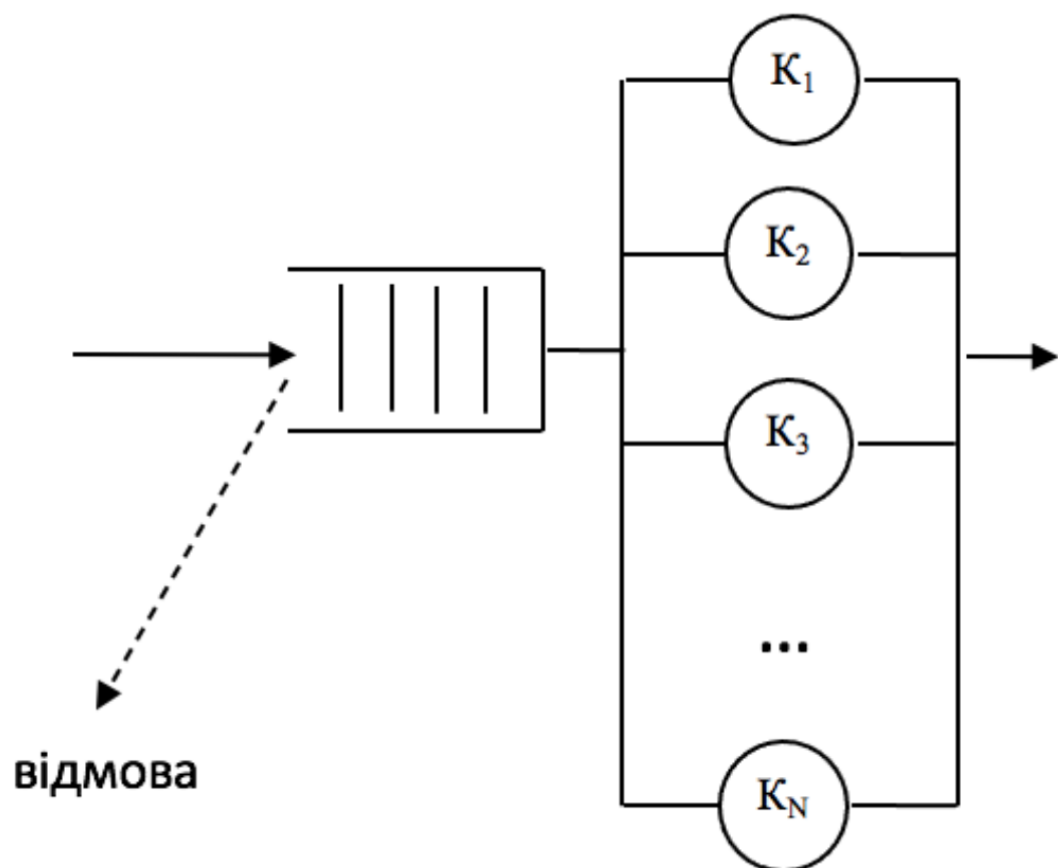


Рисунок 1.1 – Схема N-канальної СМО з відмовами

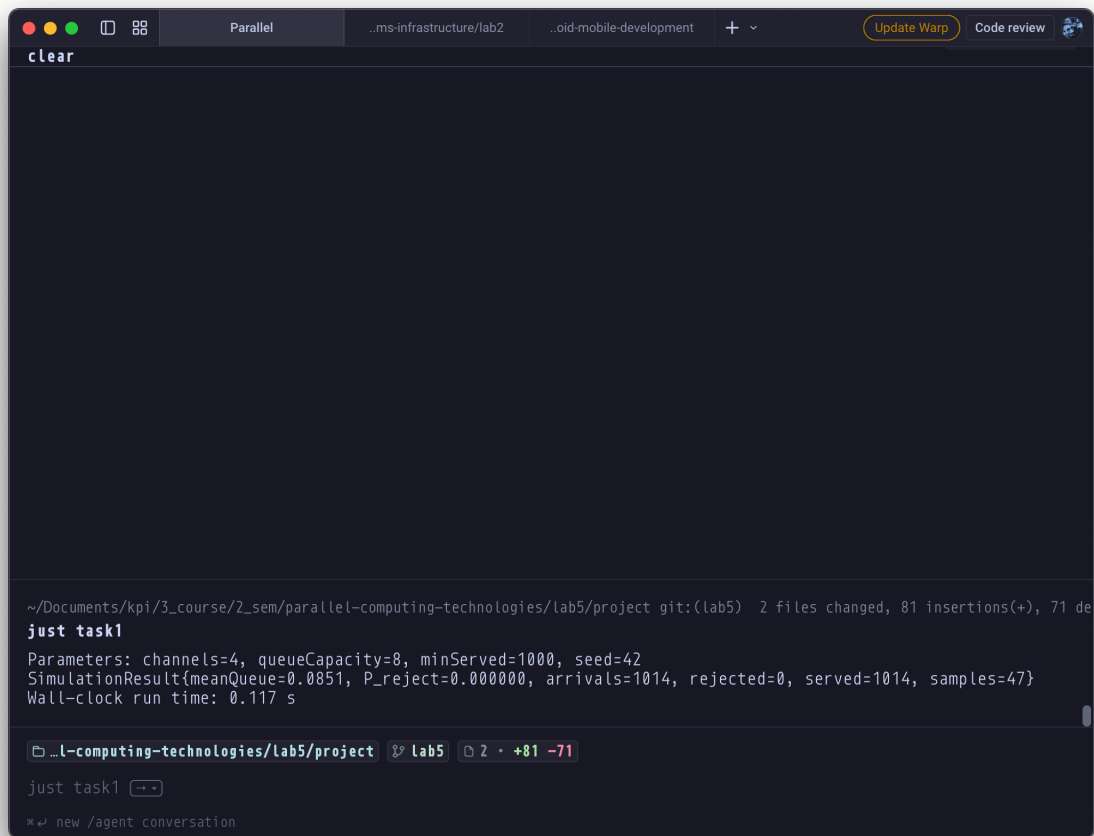


Рисунок 1.2 – Скріншот одного повного прогону імітації з типовими числовими параметрами

Таблиця 1.1 – Усереднені по реплікаціях оцінки при зміні середнього інтервалу надходження ($N=4$, $K=8$); відповідає серії `by_load` і рисунку 1.3.

meanIA, мс	Середня довжина черги	Ймовірність відмови
0.02	6.84	0.590
0.03	6.59	0.421
0.04	6.33	0.255
0.05	5.27	0.102
0.06	1.69	0.008
0.07	0.36	0.000
0.08	0.12	0.000
0.10	0.03	0.000

За рисунком 1.3 і таблицею 1.1 спостерігається різкий перехід режиму в діапазоні 0,05–0,06 мс: при меншому середньому інтервалі (вища інтенсивність надходження) система близька до насичення – великі середня довжина черги та ймовірність відмови P_{reject} ; після приблизно 0,06 мс відмови зникають, черга швидко спадає до значень близько нуля. Поблизу порогу на графіку помітніші довірчі інтервали (розкид між реплікаціями).

Таблиця 1.2 – Усереднені показники vs місткість буфера при середньому інтервалі надходження 0,05 мс ($N = 4$).

K	Середня довжина черги	Ймовірність відмови
4	1,80	0,111
8	5,29	0,104
16	12,05	0,093
24	17,99	0,080
32	24,26	0,078

За рисунком 1.4 і таблицею 1.2 збільшення K при фіксованій інтенсивності надходження підвищує середню довжину черги (більше заявок очікує в буфері замість відсікання) і знижує ймовірність відмови – зміст приблизно з 0,11 при $K = 4$ до 0,08 при $K = 32$. Зниження P_{reject} поступове; розкид оцінки відмови між реплікаціями більший, ніж для середньої довжини черги.

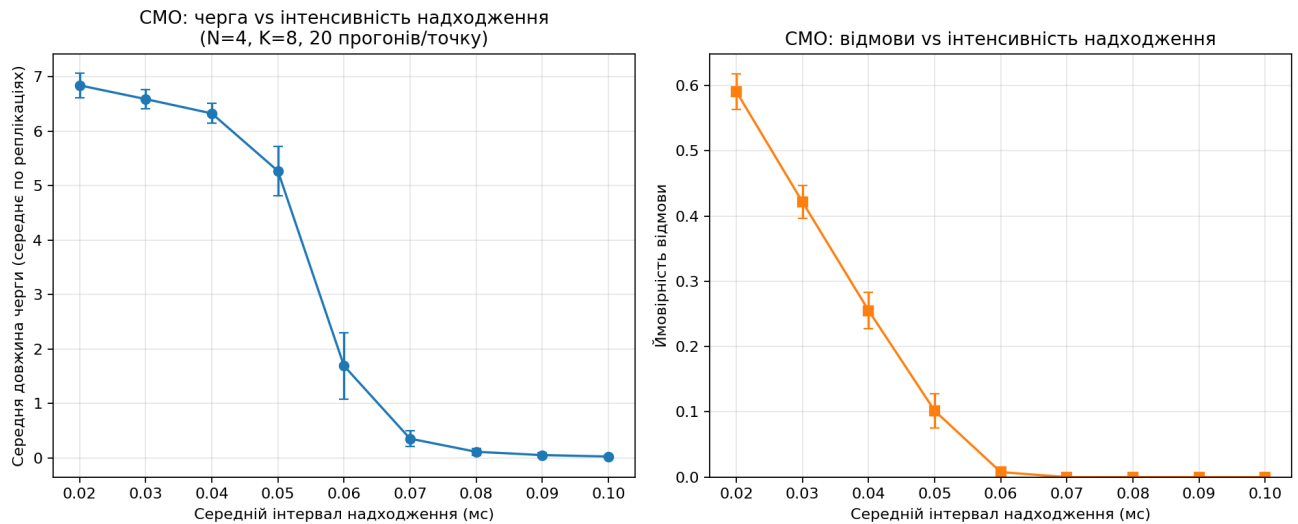


Рисунок 1.3 – Середня довжина черги та ймовірність відмови vs середній інтервал надходження (середнє та розкид по 20 реплікаціях).

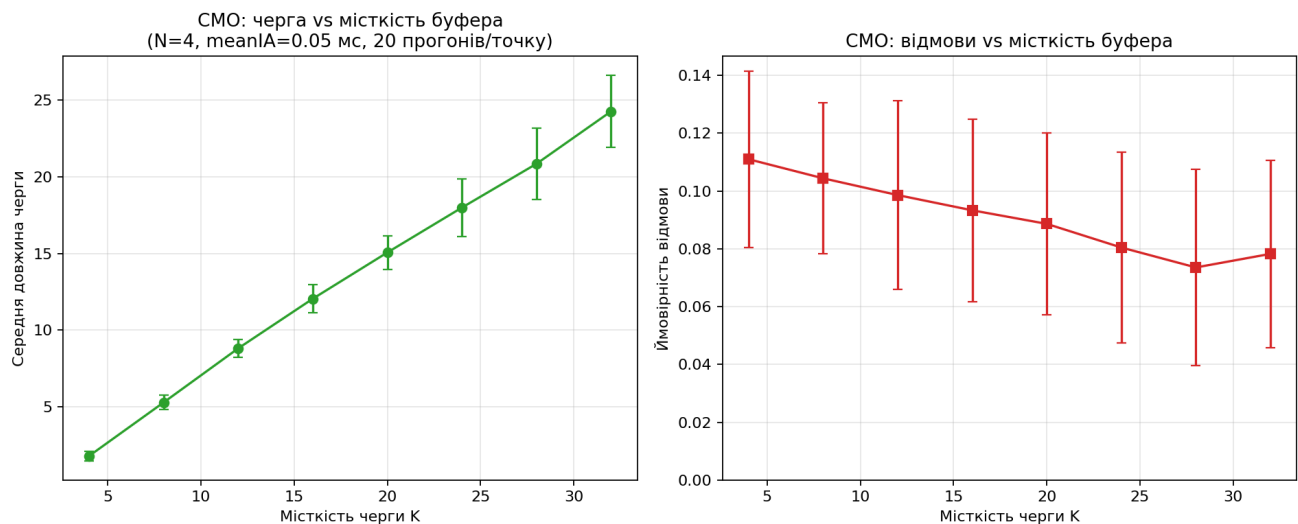


Рисунок 1.4 – Ті самі показники vs місткість черги при 0,05 мс.

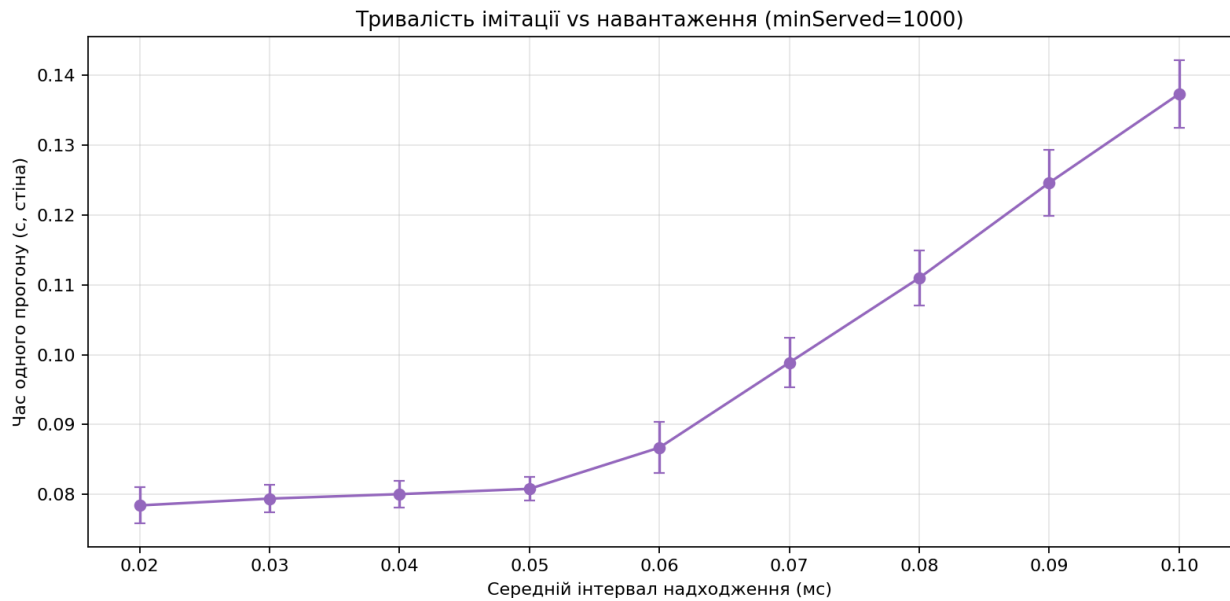


Рисунок 1.5 – Середній стіновий час одного прогону vs середній інтервал надходження

При більшому середньому інтервалі заявки надходять рідше, тому для досягнення 1000 обслугованих потрібні довший модельний час і більший реальний час прогону (наприклад, усереднено близько 0,078 с при 0,02 мс і 0,137 с при 0,10 мс). На ділянці 0,02–0,05 мс час прогону майже не змінюється, далі зростає майже лінійно – узгоджено з тим, що при сильнішому потоці заявок швидше набирається потрібна кількість обслуг.

Завдання 2

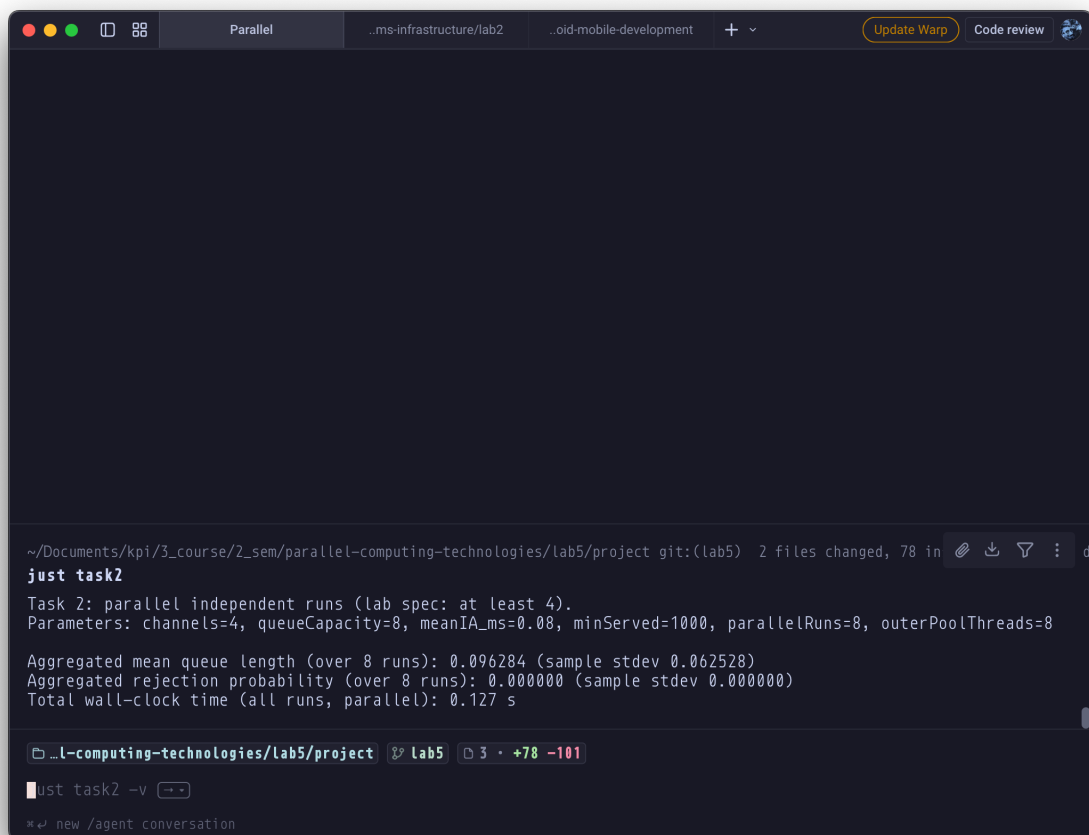
Організовано паралельне виконання кількох незалежних повних прогонів тієї ж моделі з різними випадковими насінинами; після завершення всіх прогонів обчислюють узагальнені оцінки середньої довжини черги та ймовірності відмови. За рекомендаціями до практикуму використано не менше чотирьох прогонів; у наведеному експерименті – вісім. Зовнішній рівень паралелізму обмежено кількістю доступних логічних процесорів: одночасно виконується не більше прогонів, ніж дозволяє конфігурація, решта очікує у черзі пулу. Після завершення пакета обчислюють середнє та вибіркове стандартне відхилення по серії для обох метрик і фіксують сукупний стіновий час пакета.

Таблиця 2.1 – Вісім паралельних прогонів при $N = 4$, $K = 8$, середньому інтервалі надходження 0,08 мс: середня довжина черги та ймовірність відмови по реплікаціях.

№	Середня довжина черги	Ймовірність відмови
1	0,0370	0,000000
2	0,0962	0,000000
3	0,0192	0,000000
4	0,0588	0,000000
5	0,0385	0,000000
6	0,1600	0,000000
7	0,1569	0,000000
8	0,1346	0,000000

У цьому режимі навантаження слабке (порівняно з табл. 1.1), тому у всіх восьми прогонах немає відмов ($P_{\text{reject}} = 0$). Натомість оцінка середньої довжини черги суттєво змінюється між реалізаціями (від 0,0192 до 0,1600): це наслідок стохастичності моделі та дискретного спостереження черги. Усереднення по серії дає 0,0876 зі стандартним відхиленням 0,0571 –

однієї реалізації недостатньо для стійкої характеристики; пакет із $M \geq 4$ згладжує випадкові відхилення. Сукупний час восьми паралельних прогонів порядку 0,13 с набагато менший за очікуваний час послідовного виконання тих самих восьми прогонів (порядку однієї секунди при типовій тривалості одного прогону), що ілюструє вигоду зовнішнього паралелізму для незалежних імітацій.



```
~/Documents/kpi/3_course/2_sem/parallel-computing-technologies/lab5/project git:(lab5) 2 files changed, 78 in
just task2
Task 2: parallel independent runs (lab spec: at least 4).
Parameters: channels=4, queueCapacity=8, meanIA_ms=0.08, minServed=1000, parallelRuns=8, outerPoolThreads=8

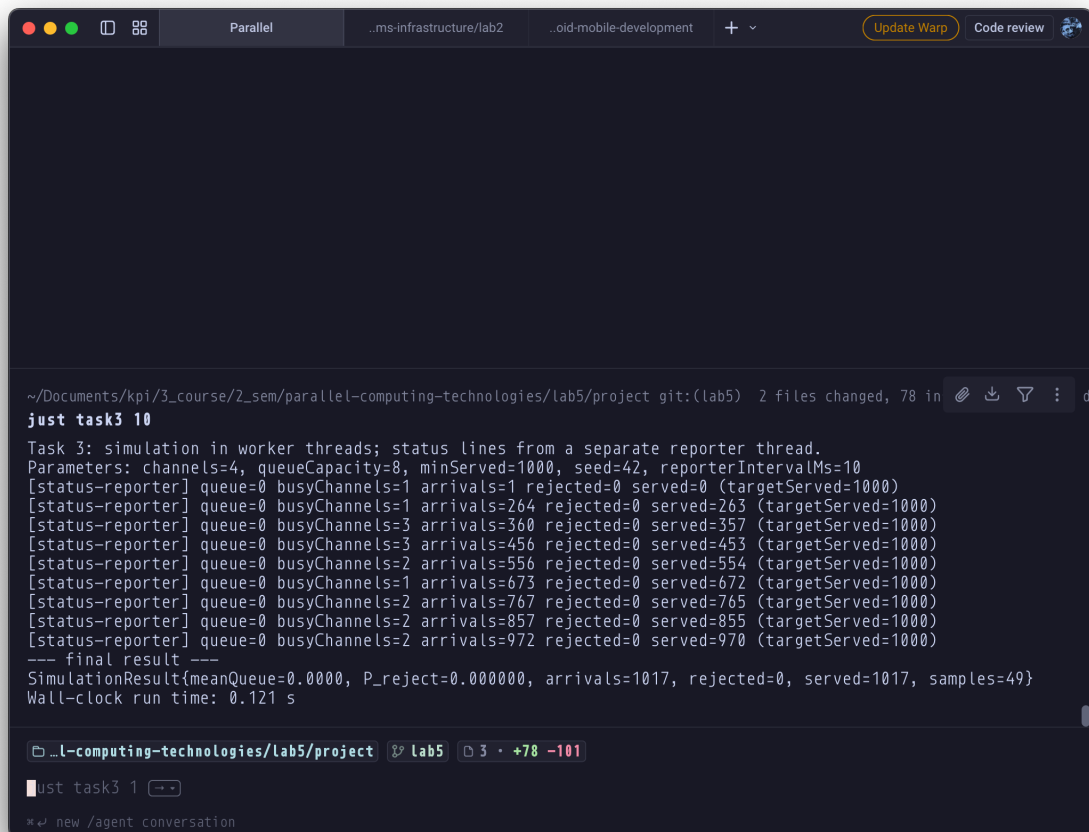
Aggregated mean queue length (over 8 runs): 0.096284 (sample stdev 0.062528)
Aggregated rejection probability (over 8 runs): 0.000000 (sample stdev 0.000000)
Total wall-clock time (all runs, parallel): 0.127 s

...l-computing-technologies/lab5/project Lab5 3 +78 -101
just task2 -v
new /agent conversation
```

Рисунок 2.1 – Результат в консолі

Завдання 3

Результати імітації (стан моделі та числові показники) виводяться в окремому потоці з заданим інтервалом оновлення, щоб спостерігати за перебігом без змішування з основним обчислювальним циклом. Періодично знімається знімок стану: довжина черги очікування, кількість зайнятих каналів, лічильники спроб входу, відмов і обслугованих заявок; доступ до спільних даних узгоджується засобами синхронізації, щоб знімок був узгодженим. Вивід рядків статусу виконується з невеликим інтервалом (за замовчуванням порядку 100 мс), з можливістю зміни інтервалу для зручності спостереження; перший знімок формується одразу після старту спостереження, далі – після кожної паузи, щоб при короткому прогоні встигло з’явитися кілька рядків.



```
~/Documents/kpi/3_course/2_sem/parallel-computing-technologies/lab5/project git:(lab5) 2 files changed, 78 in
just task3 10
Task 3: simulation in worker threads; status lines from a separate reporter thread.
Parameters: channels=4, queueCapacity=8, minServed=1000, seed=42, reporterIntervalMs=10
[status-reporter] queue=0 busyChannels=1 arrivals=1 rejected=0 served=0 (targetServed=1000)
[status-reporter] queue=0 busyChannels=1 arrivals=264 rejected=0 served=263 (targetServed=1000)
[status-reporter] queue=0 busyChannels=3 arrivals=360 rejected=0 served=357 (targetServed=1000)
[status-reporter] queue=0 busyChannels=3 arrivals=456 rejected=0 served=453 (targetServed=1000)
[status-reporter] queue=0 busyChannels=2 arrivals=556 rejected=0 served=554 (targetServed=1000)
[status-reporter] queue=0 busyChannels=1 arrivals=673 rejected=0 served=672 (targetServed=1000)
[status-reporter] queue=0 busyChannels=2 arrivals=767 rejected=0 served=765 (targetServed=1000)
[status-reporter] queue=0 busyChannels=2 arrivals=857 rejected=0 served=855 (targetServed=1000)
[status-reporter] queue=0 busyChannels=2 arrivals=972 rejected=0 served=970 (targetServed=1000)
--- final result ---
SimulationResult{meanQueue=0.0000, P_reject=0.000000, arrivals=1017, rejected=0, served=1017, samples=49}
Wall-clock run time: 0.121 s
```

Рисунок 3.1 – Результат в консолі

Завдання 4

Обрано алгоритм завдання 2 цього практикуму: паралельний пакет із M незалежних прогонів імітації СМО з подальшою агрегацією оцінок. Показники S_p , E_p і оцінка за Амдалом стосуються зовнішнього рівня (паралельне виконання кількох повних прогонів), а не внутрішнього паралелізму каналів однієї СМО (завдання 1).

Припущення.

1. Прогони незалежні; в середньому однакової обчислювальної вартості (ті самі параметри зупинки та навантаження).
2. Доступно p робочих потоків зовнішнього пулу, $p = \min(M, P)$, де P – кількість доступних логічних процесорів. Якщо $M \leq p$, усі прогони можуть виконуватися одночасно; якщо $M > p$, одночасно працює не більше p прогонів, решта – у черзі.
3. Послідовний еталон: $T_{\text{seq}} \approx M\bar{t}$, де $\bar{t}$ – типовий час одного прогону.
4. Накладні витрати на планування пакета та збір результатів малі порівняно з $M\bar{t}$; частка послідовної роботи в сенсі Амдала $f \ll 1$.

Прискорення та ефективність при p потоках:

$$S_p = \frac{T_{\text{seq}}}{T_{\text{par}}}, \quad E_p = \frac{S_p}{p}.$$

Закон Амдала: якщо f – частка непаралелізованої роботи, а $(1 - f)$ ідеально масштабується на p потоках, то

$$S_p \leq S_{\text{Amdahl}}(p) = \frac{1}{f + \frac{1-f}{p}}.$$

Для пакета незалежних прогонів f близька до нуля, тому $S_{\text{Amdahl}}(p) \approx p$ при дуже малих f . Обмеження: кожен прогон внутрішньо багатопотоковий (канали, надходження, спостереження за чергою), тому на

обмеженій кількості фізичних ядер можливе переповнення планувальника; тоді T_{par} зростає, а фактичне S_p буває меншим за ідеальне.

Порівняння з експериментом (завд. 2). Для $M = 8$ і $p = 8$ вимірний сукупний час пакета $T_{\text{par}} \approx 0,13$ с, один прогон у тому ж режимі – порядку 0,12–0,13 с. Оцінка $T_{\text{seq}} \approx 8\bar{t} \sim 1$ с дає $S_p \sim T_{\text{seq}}/T_{\text{par}}$ порядку 6–8, $E_p = S_p/p$ близько 0,8–1,0 – узгоджено з очікуванням для незалежних задач при $M \leq p$. Якщо $P < 8$, маємо $p < P$ і більше «шарів» виконання по $\bar{t}$, що відповідає обмеженій паралельності пулу.

Таблиця 4.1 – Теоретичні залежності та орієнтири за експериментом ($M = 8$, однакові параметри СМО).

Показник	Теоретична залежність	Орієнтир (завд. 2)
T_{par}	$\approx \bar{t}$	$\sim 0,13$ с
при $M \leq p$		
T_{seq}	$\approx M\bar{t}$	$\sim 8 \cdot 0,13$ с ≈ 1 с
S_p	$\leq S_{\text{Amdahl}}(p)$; при малому f та $M \leq p$ – порядок $\min(M, p)$	порядку 6–8
E_p	S_p/p	близько 0,8–1,0 при $p \approx 8$

Висновок

У межах практикуму №5 розглянуто багатоканальну СМО з обмеженою чергою та відмовами, реалізовано імітацію з пулом потоків і окремим спостереженням за довжиною черги; ймовірність відмови визначається з лічильників спроб і відмов. Експериментальні ряди (табл. 1.1–1.2, рис. 1.3–1.5) показують вплив інтенсивності надходження, місткості буфера та критерію зупинки на метрики системи. Паралельні незалежні прогони (завд. 2) підвищують надійність статистичних оцінок; періодичний вивід стану (завд. 3) робить перебіг моделі наочним. Теоретичний аналіз (завд. 4) зв'язує виміряне прискорення пакета прогонів із законом Амдала та показниками S_p , E_p (табл. 4.1, рис. 4.1). Практикум ілюструє роль багатопотокових засобів високого рівня при імітаційному моделюванні та необхідність коректної синхронізації спільного стану.

Код програми

Посилання

на

GitHub

репозиторій:

<https://github.com/sokolovgit/parallel-computing-technologies>

```
=== lab5/common/Randoms.java ===
package lab5.common;

import java.util.Random;
import java.util.concurrent.ThreadLocalRandom;

/**
 * Random inter-arrival times (uniform with a given mean) and service
 * times (normal, clamped).
 */
public final class Randoms {

    private Randoms() {}

    /**
     * Uniform inter-arrival on [0, 2*mean], so the expected value equals
     * {@code mean}.
     */
    public static long nextInterArrivalNanos(Random rng, long
meanInterArrivalNanos) {
        if (meanInterArrivalNanos < 1) {
            return 1;
        }
        double u = rng.nextDouble();
        long v = (long) (u * 2.0 * meanInterArrivalNanos);
        return Math.max(1L, v);
    }

    public static long nextServiceNanos(SimulationConfig cfg, Random rng)
{
        double g = rng.nextGaussian();
        return clampServiceNanos(cfg, g);
    }

    /** For pool worker threads: {@link ThreadLocalRandom} (Java 8 TLR has
no {@code nextGaussian}). */
    public static long nextServiceNanos(SimulationConfig cfg) {
        double g = nextGaussian(ThreadLocalRandom.current());
        return clampServiceNanos(cfg, g);
    }

    private static long clampServiceNanos(SimulationConfig cfg, double g)
{
        long v = (long) (cfg.serviceMeanNanos() + g *
cfg.serviceStdDevNanos());
        if (v < cfg.serviceMinNanos()) {
            v = cfg.serviceMinNanos();
        }
        if (v > cfg.serviceMaxNanos()) {
            v = cfg.serviceMaxNanos();
        }
        return v;
    }
}
```

```

    }

    private static double nextGaussian(ThreadLocalRandom r) {
        double u1;
        do {
            u1 = r.nextDouble();
        } while (u1 <= 1e-12);
        double u2 = r.nextDouble();
        return Math.sqrt(-2.0 * Math.log(u1)) * Math.cos(2.0 * Math.PI *
u2);
    }
}

=== lab5/common/SimulationConfig.java ===
package lab5.common;

/**
 * Parameters for one simulation run of a multi-server queueing system
 * with a finite buffer.
 */
public final class SimulationConfig {

    private final int channels;
    private final int queueCapacity;
    /** Mean inter-arrival time (ns); implemented as Uniform[0, 2*mean] so
E[interval] = mean. */
    private final long meanInterArrivalNanos;
    /** Normal service-time parameters (ns); values are clamped to a
positive range. */
    private final long serviceMeanNanos;
    private final long serviceStdDevNanos;
    private final long serviceMinNanos;
    private final long serviceMaxNanos;
    /** Minimum number of served customers before stopping the run (lab
spec: at least 1000). */
    private final int minServed;
    /** Queue-length sampling period (ms), observed in a dedicated thread.
*/
    private final long queueSampleIntervalMillis;
    private final long randomSeed;

    public SimulationConfig(
        int channels,
        int queueCapacity,
        long meanInterArrivalNanos,
        long serviceMeanNanos,
        long serviceStdDevNanos,
        long serviceMinNanos,
        long serviceMaxNanos,
        int minServed,
        long queueSampleIntervalMillis,
        long randomSeed) {
        if (channels < 1) {
            throw new IllegalArgumentException("channels must be >= 1");
        }
        if (queueCapacity < 0) {
            throw new IllegalArgumentException("queueCapacity must be >=
0");

```



```

    }
    if (meanInterArrivalNanos < 1) {
        throw new IllegalArgumentException("meanInterArrivalNanos must
be >= 1");
    }
    if (minServed < 1) {
        throw new IllegalArgumentException("minServed must be >= 1");
    }
    if (queueSampleIntervalMillis < 1) {
        throw new IllegalArgumentException("queueSampleIntervalMillis
must be >= 1");
    }
    this.channels = channels;
    this.queueCapacity = queueCapacity;
    this.meanInterArrivalNanos = meanInterArrivalNanos;
    this.serviceMeanNanos = serviceMeanNanos;
    this.serviceStdDevNanos = serviceStdDevNanos;
    this.serviceMinNanos = serviceMinNanos;
    this.serviceMaxNanos = serviceMaxNanos;
    this.minServed = minServed;
    this.queueSampleIntervalMillis = queueSampleIntervalMillis;
    this.randomSeed = randomSeed;
}

```

/** Factory that takes times in milliseconds and converts them to nanoseconds. */

```

public static SimulationConfig fromMillis(
    int channels,
    int queueCapacity,
    double meanInterArrivalMs,
    double serviceMeanMs,
    double serviceStdDevMs,
    int minServed,
    long queueSampleIntervalMillis,
    long randomSeed) {
    long msToNs = 1_000_000L;
    return new SimulationConfig(
        channels,
        queueCapacity,
        Math.max(1L, (long) (meanInterArrivalMs * msToNs)),
        (long) (serviceMeanMs * msToNs),
        Math.max(1L, (long) (serviceStdDevMs * msToNs)),
        1L,
        Math.max(1L, (long) (serviceMeanMs * 5 * msToNs)),
        minServed,
        queueSampleIntervalMillis,
        randomSeed);
}

```

```

public int channels() {
    return channels;
}

```

```

public int queueCapacity() {
    return queueCapacity;
}

```

```

public long meanInterArrivalNanos() {

```

```

        return meanInterArrivalNanos;
    }

    public long serviceMeanNanos() {
        return serviceMeanNanos;
    }

    public long serviceStdDevNanos() {
        return serviceStdDevNanos;
    }

    public long serviceMinNanos() {
        return serviceMinNanos;
    }

    public long serviceMaxNanos() {
        return serviceMaxNanos;
    }

    public int minServed() {
        return minServed;
    }

    public long queueSampleIntervalMillis() {
        return queueSampleIntervalMillis;
    }

    public long randomSeed() {
        return randomSeed;
    }
}

```

```

=== lab5/common/SimulationResult.java ===
package lab5.common;

```

```

/**
 * Outcome of one run: mean queue length (from discrete samples) and
 * rejection probability.
 */
public final class SimulationResult {

    private final double meanQueueLength;
    private final double rejectionProbability;
    private final long arrivalAttempts;
    private final long rejected;
    private final long served;
    private final long queueSampleCount;

    public SimulationResult(
        double meanQueueLength,
        double rejectionProbability,
        long arrivalAttempts,
        long rejected,
        long served,
        long queueSampleCount) {
        this.meanQueueLength = meanQueueLength;
        this.rejectionProbability = rejectionProbability;
        this.arrivalAttempts = arrivalAttempts;
    }
}

```

```

        this.rejected = rejected;
        this.served = served;
        this.queueSampleCount = queueSampleCount;
    }

    public double meanQueueLength() {
        return meanQueueLength;
    }

    public double rejectionProbability() {
        return rejectionProbability;
    }

    public long arrivalAttempts() {
        return arrivalAttempts;
    }

    public long rejected() {
        return rejected;
    }

    public long served() {
        return served;
    }

    public long queueSampleCount() {
        return queueSampleCount;
    }

    @Override
    public String toString() {
        return String.format(
            "SimulationResult{meanQueue=%.4f, P_reject=%.6f,
arrivals=%d, rejected=%d, served=%d, samples=%d}",
            meanQueueLength,
            rejectionProbability,
            arrivalAttempts,
            rejected,
            served,
            queueSampleCount);
    }
}

```

```

=== lab5/smo/Customer.java ===
package lab5.smo;

```

```

/** Customer request in the queueing system (id for tracing). */
final class Customer {

    private final long id;

    Customer(long id) {
        this.id = id;
    }

    long id() {
        return id;
    }
}

```

```
}
```

```
=== lab5/smo/MultiChannelSmo.java ===  
package lab5.smo;
```

```
import java.util.ArrayDeque;  
import java.util.Queue;  
import java.util.Random;  
import java.util.concurrent.ExecutorService;  
import java.util.concurrent.Executors;  
import java.util.concurrent.TimeUnit;  
import java.util.concurrent.atomic.AtomicBoolean;  
import java.util.concurrent.atomic.AtomicLong;  
import java.util.concurrent.locks.LockSupport;
```

```
import lab5.common.Randoms;  
import lab5.common.SimulationConfig;  
import lab5.common.SimulationResult;
```

```
/**  
 * Simulation of an N-server queueing system with a finite buffer and  
rejections.  
 * Each service is run as a task on a fixed {@link ExecutorService} (one  
thread pool worker per server).  
 */
```

```
public final class MultiChannelSmo {  
  
    /** Point-in-time view of the model for live monitoring (task 3). */  
    public static final class SmoSnapshot {  
  
        private final int queueSize;  
        private final int busyChannels;  
        private final long arrivalAttempts;  
        private final long rejected;  
        private final long served;  
  
        public SmoSnapshot(  
            int queueSize,  
            int busyChannels,  
            long arrivalAttempts,  
            long rejected,  
            long served) {  
            this.queueSize = queueSize;  
            this.busyChannels = busyChannels;  
            this.arrivalAttempts = arrivalAttempts;  
            this.rejected = rejected;  
            this.served = served;  
        }  
  
        public int queueSize() {  
            return queueSize;  
        }  
  
        public int busyChannels() {  
            return busyChannels;  
        }  
  
        public long arrivalAttempts() {
```

```

        return arrivalAttempts;
    }

    public long rejected() {
        return rejected;
    }

    public long served() {
        return served;
    }
}

private final SimulationConfig cfg;
private final Object lock = new Object();
private final Queue<Customer> waitQueue = new ArrayDeque<>();
private final ExecutorService channelPool;
private final Random arrivalRng;

private int busyChannels;
private final AtomicLong arrivalAttempts = new AtomicLong();
private final AtomicLong rejected = new AtomicLong();
private final AtomicLong served = new AtomicLong();

private final AtomicBoolean stopProducing = new AtomicBoolean();
private final AtomicBoolean samplingActive = new AtomicBoolean(true);

private final AtomicLong queueSampleSum = new AtomicLong();
private final AtomicLong queueSampleCount = new AtomicLong();

public MultiChannelSmo(SimulationConfig cfg) {
    this.cfg = cfg;
    this.channelPool = Executors.newFixedThreadPool(cfg.channels());
    this.arrivalRng = new Random(cfg.randomSeed());
}

/**
 * Consistent view of queue length, busy servers, and counters (for a
 * separate status thread).
 */
public SmoSnapshot snapshot() {
    synchronized (lock) {
        return new SmoSnapshot(
            waitQueue.size(),
            busyChannels,
            arrivalAttempts.get(),
            rejected.get(),
            served.get());
    }
}

public SimulationResult run() throws InterruptedException {
    return runSimulation(0L);
}

/**
 * Same as {@link #run}, but a daemon {@code status-reporter} thread
 * prints {@link #snapshot()} every

```

```

        * {@code reportIntervalMillis} (lab task 3: dynamic output in a
        separate thread).
        */
        public SimulationResult runWithLiveReporter(long reportIntervalMillis)
        throws InterruptedException {
            if (reportIntervalMillis < 1L) {
                throw new IllegalArgumentException("reportIntervalMillis must
                be >= 1");
            }
            return runSimulation(reportIntervalMillis);
        }

        private SimulationResult runSimulation(long reportIntervalMillis)
        throws InterruptedException {
            boolean withReporter = reportIntervalMillis > 0L;

            Thread sampler = new Thread(this::sampleQueueLoop, "queue-
            sampler");
            sampler.setDaemon(true);
            sampler.start();

            Thread producer = new Thread(this::producerLoop, "arrival-
            producer");
            producer.start();

            final AtomicBoolean reporterActive = new
            AtomicBoolean(withReporter);
            Thread reporter = null;
            if (withReporter) {
                final long period = reportIntervalMillis;
                reporter =
                    new Thread(
                        () -> {
                            // Print first, then sleep – otherwise a
                            ~0.1 s run with a 50 ms period
                            // only shows one line (first tick was
                            after the full sleep).
                            while (reporterActive.get()) {
                                SmoSnapshot s = snapshot();
                                synchronized (System.out) {
                                    System.out.printf(
                                        "[status-reporter]
                                        queue=%d busyChannels=%d "
                                        rejected=%d served=%d (targetServed=%d)%n",
                                        s.queueSize(),
                                        s.busyChannels(),
                                        s.arrivalAttempts(),
                                        s.rejected(),
                                        s.served(),
                                        cfg.minServed());
                                }
                            }
                            if (!reporterActive.get()) {
                                break;
                            }
                        }
                    );
            }
            try {
                Thread.sleep(period);
            } catch (InterruptedException e) {

```

```

Thread.currentThread().interrupt();
                                break;
                                }
                                },
                                "status-reporter");
    reporter.setDaemon(true);
    reporter.start();
}

try {
    long deadlineNanos = System.nanoTime() +
TimeUnit.MINUTES.toNanos(10);
    while (served.get() < cfg.minServed()) {
        if (System.nanoTime() > deadlineNanos) {
            stopProducing.set(true);
            producer.join();
            channelPool.shutdownNow();
            samplingActive.set(false);
            sampler.join(5_000);
            throw new IllegalStateException(
                "Could not reach minServed="
                    + cfg.minServed()
                    + " within 10 minutes; reduce load or
add queue capacity / servers.");
        }
        Thread.sleep(2L);
    }
    stopProducing.set(true);
    producer.join();

    waitUntilDrained();

    channelPool.shutdown();
    boolean finished = channelPool.awaitTermination(5,
TimeUnit.MINUTES);
    if (!finished) {
        channelPool.shutdownNow();
    }

    samplingActive.set(false);
    sampler.join(5_000);
} finally {
    reporterActive.set(false);
    if (reporter != null) {
        reporter.interrupt();
        reporter.join(2_000);
    }
}

    long samples = queueSampleCount.get();
    double meanQueue = samples == 0 ? 0.0 : queueSampleSum.get() /
(double) samples;
    long attempts = arrivalAttempts.get();
    double pReject = attempts == 0 ? 0.0 : rejected.get() / (double)
attempts;

```

```

        return new SimulationResult(
            meanQueue,
            pReject,
            attempts,
            rejected.get(),
            served.get(),
            samples);
    }

    /**
     * Waits until no customer is in service and the waiting buffer is
     empty, so no further
     * {@code execute} calls will be scheduled before {@link
     ExecutorService#shutdown()}.
     */
    private void waitUntilDrained() throws InterruptedException {
        long deadlineNanos = System.nanoTime() +
            TimeUnit.MINUTES.toNanos(5);
        while (true) {
            synchronized (lock) {
                if (waitQueue.isEmpty() && busyChannels == 0) {
                    return;
                }
            }
            if (System.nanoTime() > deadlineNanos) {
                throw new IllegalStateException(
                    "Queue/system did not drain within 5 minutes after
stopping arrivals.");
            }
            Thread.sleep(1L);
        }
    }

    private void producerLoop() {
        try {
            while (!stopProducing.get()) {
                long waitNs = Randoms.nextInterArrivalNanos(arrivalRng,
                    cfg.meanInterArrivalNanos());
                LockSupport.parkNanos(waitNs);
                admitOneArrival();
            }
        } catch (Throwable ignored) {
            // interruption or other fault – stop generating arrivals
        }
    }

    private void admitOneArrival() {
        Customer start = null;
        synchronized (lock) {
            long n = arrivalAttempts.incrementAndGet();
            if (busyChannels < cfg.channels()) {
                busyChannels++;
                start = new Customer(n);
            } else if (waitQueue.size() < cfg.queueCapacity()) {
                waitQueue.add(new Customer(n));
            } else {
                rejected.incrementAndGet();
            }
        }
    }

```



```

    }
    final Customer toServe = start;
    if (toServe != null) {
        channelPool.execute(() -> serveCustomer(toServe));
    }
}

private void serveCustomer(Customer customer) {
    long ns = Randoms.nextServiceNanos(cfg);
    LockSupport.parkNanos(ns);

    Customer next = null;
    synchronized (lock) {
        busyChannels--;
        served.incrementAndGet();
        if (!waitQueue.isEmpty()) {
            next = waitQueue.poll();
            busyChannels++;
        }
    }
    final Customer toServeNext = next;
    if (toServeNext != null) {
        channelPool.execute(() -> serveCustomer(toServeNext));
    }
}

private void sampleQueueLoop() {
    long intervalMs = cfg.queueSampleIntervalMillis();
    while (samplingActive.get()) {
        try {
            Thread.sleep(intervalMs);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            break;
        }
        int len;
        synchronized (lock) {
            len = waitQueue.size();
        }
        queueSampleSum.addAndGet(len);
        queueSampleCount.incrementAndGet();
    }
}
}

```

=== lab5/task1/Task1Benchmark.java ===

```

package lab5.task1;

import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

import lab5.common.SimulationConfig;
import lab5.common.SimulationResult;
import lab5.smo.MultiChannelSmo;

```

```

/**
 * Batch runs for plotting: sweeps mean inter-arrival time and queue
capacity; writes
 * {@code results/lab5_by_load.csv} and {@code results/lab5_by_queue.csv}.
Invoked by {@code just task1-plots}.
 */
public final class Task1Benchmark {

    private static final String CSV_HEADER =

"sweep,meanIA_ms,channels,queue,rep,seed,meanQueue,P_reject,arrivals,rejec
ted,served,qSamples,wall_s";

    private static final int MIN_SERVED = 1000;
    private static final long SAMPLE_INTERVAL_MS = 2L;
    private static final int CHANNELS = 4;
    private static final int DEFAULT_QUEUE = 8;
    private static final double SERV_MEAN_MS = 0.25;
    private static final double SERV_STD_MS = 0.06;
    private static final int REPS_PER_POINT = 20;

    private Task1Benchmark() {}

    public static void main(String[] args) throws Exception {
        String home = System.getProperty("lab5.home", ".");
        Path base = Paths.get(home).normalize();
        Path results = base.resolve("results");
        Files.createDirectories(results);

        Path byLoad = results.resolve("lab5_by_load.csv");
        Path byQueue = results.resolve("lab5_by_queue.csv");

        System.err.println("Writing " + byLoad + " (this may take a
minute)...");
        runLoadSweep(byLoad);
        System.err.println("Writing " + byQueue + "...");
        runQueueSweep(byQueue);
        System.err.println("Done.");
    }

    /** Vary mean inter-arrival time (ms); lower = heavier load. */
    private static void runLoadSweep(Path out) throws Exception {
        try (PrintWriter w = new PrintWriter(Files.newBufferedWriter(out,
StandardCharsets.UTF_8))) {
            w.println(CSV_HEADER);
            for (double meanIa = 0.02; meanIa <= 0.1000001; meanIa +=
0.01) {
                double ia = Math.round(meanIa * 100.0) / 100.0;
                for (int rep = 0; rep < REPS_PER_POINT; rep++) {
                    long seed = mixSeed(CHANNELS, DEFAULT_QUEUE, ia, rep,
1);
                    writeOneRun(w, "by_load", ia, CHANNELS, DEFAULT_QUEUE,
rep, seed);
                }
            }
        }
    }
}

```

```

    /** Vary queue capacity at fixed moderate load. */
    private static void runQueueSweep(Path out) throws Exception {
        double meanIa = 0.05;
        try (PrintWriter w = new PrintWriter(Files.newBufferedWriter(out,
            StandardCharsets.UTF_8))) {
            w.println(CSV_HEADER);
            for (int queue = 4; queue <= 32; queue += 4) {
                for (int rep = 0; rep < REPS_PER_POINT; rep++) {
                    long seed = mixSeed(CHANNELS, queue, meanIa, rep, 2);
                    writeOneRun(w, "by_queue", meanIa, CHANNELS, queue,
rep, seed);
                }
            }
        }

        private static long mixSeed(int channels, int queue, double meanIa,
int rep, int salt) {
            long h = salt * 0x9E3779B97F4A7C15L;
            h ^= (long) channels << 32;
            h ^= queue;
            h ^= Double.doubleToRawLongBits(meanIa);
            h ^= (long) rep * 0x85EBCA6B;
            h *= 0xC2B2AE3D27D4EB4FL;
            return h == 0 ? 1L : h;
        }

        private static void writeOneRun(
            PrintWriter w,
            String sweep,
            double meanIaMs,
            int channels,
            int queue,
            int rep,
            long seed)
            throws Exception {
            SimulationConfig cfg =
                SimulationConfig.fromMillis(
                    channels,
                    queue,
                    meanIaMs,
                    SERV_MEAN_MS,
                    SERV_STD_MS,
                    MIN_SERVED,
                    SAMPLE_INTERVAL_MS,
                    seed);

            long t0 = System.nanoTime();
            SimulationResult r = new MultiChannelSmo(cfg).run();
            double wallS = (System.nanoTime() - t0) / 1_000_000_000.0;

            w.printf(
                "%s,%.4f,%d,%d,%d,%d,%.8f,%.8f,%d,%d,%d,%d,%.6f%n",
                sweep,
                meanIaMs,
                channels,
                queue,
                rep,

```

```

        seed,
        r.meanQueueLength(),
        r.rejectionProbability(),
        r.arrivalAttempts(),
        r.rejected(),
        r.served(),
        r.queueSampleCount(),
        walls);
    w.flush();
}
}

```

=== lab5/task1/Task1Main.java ===

```
package lab5.task1;
```

```
import lab5.common.SimulationConfig;
import lab5.common.SimulationResult;
import lab5.smo.MultiChannelSmo;
```

```
/**
```

```
 * Lab 5, part 1: single run of an N-server queueing model (thread pool =
servers),
```

```
 * mean queue length and rejection probability.
```

```
 */
```

```
public final class Task1Main {
```

```
    private Task1Main() {}
```

```
    public static void main(String[] args) throws Exception {
```

```
        SimulationConfig cfg =
```

```
            SimulationConfig.fromMillis(
```

```
                4,
```

```
                8,
```

```
                0.08,
```

```
                0.25,
```

```
                0.06,
```

```
                1000,
```

```
                2L,
```

```
                42L);
```

```
        System.out.println("Parameters: channels=" + cfg.channels()
```

```
            + ", queueCapacity=" + cfg.queueCapacity()
```

```
            + ", minServed=" + cfg.minServed()
```

```
            + ", seed=" + cfg.randomSeed());
```

```
        MultiChannelSmo smo = new MultiChannelSmo(cfg);
```

```
        long t0 = System.nanoTime();
```

```
        SimulationResult r = smo.run();
```

```
        long dt = System.nanoTime() - t0;
```

```
        System.out.println(r);
```

```
        System.out.printf("Wall-clock run time: %.3f s%n", dt /
```

```
1_000_000_000.0);
```

```
    }
```

```
}
```

=== lab5/task2/Task2Main.java ===

```
package lab5.task2;
```

```

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;

import lab5.common.SimulationConfig;
import lab5.common.SimulationResult;
import lab5.smo.MultiChannelSmo;

/**
 * Lab 5, part 2: run several independent SMO simulations in parallel
 * (separate {@link MultiChannelSmo}
 * * per replication, distinct seeds) and aggregate mean queue length and
 * rejection probability.
 */
public final class Task2Main {

    private static final int MIN_RUNS = 4;
    private static final int DEFAULT_RUNS = 8;
    private static final long BASE_SEED = 42L;

    private Task2Main() {}

    public static void main(String[] args) throws Exception {
        int runs = parseRuns(args);
        boolean verbose = hasFlag(args, "-v", "--verbose");

        int channels = 4;
        int queueCapacity = 8;
        double meanIaMs = 0.08;
        double servMeanMs = 0.25;
        double servStdMs = 0.06;
        int minServed = 1000;
        long sampleIntervalMs = 2L;

        int cores = Runtime.getRuntime().availableProcessors();
        int poolSize = Math.max(1, Math.min(runs, cores));

        System.out.println("Task 2: parallel independent runs (lab spec:
at least " + MIN_RUNS + ").");
        System.out.println("Parameters: channels=" + channels + ",
queueCapacity=" + queueCapacity
        + ", meanIA_ms=" + meanIaMs + ", minServed=" + minServed +
", parallelRuns=" + runs
        + ", outerPoolThreads=" + poolSize);

        List<Callable<SimulationResult>> tasks = new ArrayList<>();
        for (int i = 0; i < runs; i++) {
            final long seed = BASE_SEED + (long) i * 0x9E3779B97F4A7C15L;
            tasks.add(
                () -> {
                    SimulationConfig cfg =
                        SimulationConfig.fromMillis(

```

```

        channels,
        queueCapacity,
        meanIaMs,
        servMeanMs,
        servStdMs,
        minServed,
        sampleIntervalMs,
        seed);
    return new MultiChannelSmo(cfg).run();
});
}

ExecutorService outer = Executors.newFixedThreadPool(poolSize);
long t0 = System.nanoTime();
List<Future<SimulationResult>> futures;
try {
    futures = outer.invokeAll(tasks);
} finally {
    outer.shutdown();
    if (!outer.awaitTermination(30, TimeUnit.MINUTES)) {
        outer.shutdownNow();
    }
}
long wallNs = System.nanoTime() - t0;

double[] meanQueues = new double[runs];
double[] pRejects = new double[runs];
for (int i = 0; i < runs; i++) {
    try {
        SimulationResult r = futures.get(i).get();
        meanQueues[i] = r.meanQueueLength();
        pRejects[i] = r.rejectionProbability();
        if (verbose) {
            System.out.println("run " + i + " seed=" + (BASE_SEED
+ (long) i * 0x9E3779B97F4A7C15L) + " -> " + r);
        }
    } catch (ExecutionException e) {
        Throwable c = e.getCause() != null ? e.getCause() : e;
        throw new RuntimeException("run " + i + " failed: " +
c.getMessage(), c);
    }
}

double mqMean = mean(meanQueues);
double mqStd = sampleStdDev(meanQueues);
double prMean = mean(pRejects);
double prStd = sampleStdDev(pRejects);

System.out.println();
System.out.printf("Aggregated mean queue length (over %d runs):
%.6f (sample stdev %.6f)%n", runs, mqMean, mqStd);
System.out.printf("Aggregated rejection probability (over %d
runs): %.6f (sample stdev %.6f)%n", runs, prMean, prStd);
System.out.printf("Total wall-clock time (all runs, parallel):
%.3f s%n", wallNs / 1_000_000_000.0);
}

private static int parseRuns(String[] args) {

```

```

        for (String a : args) {
            if (a.startsWith("-")) {
                continue;
            }
            try {
                int n = Integer.parseInt(a);
                if (n < MIN_RUNS) {
                    System.err.println("runs must be >= " + MIN_RUNS + ",
using " + MIN_RUNS);
                    return MIN_RUNS;
                }
                return n;
            } catch (NumberFormatException ignored) {
                System.err.println("Ignoring non-integer argument: " + a);
            }
        }
        return DEFAULT_RUNS;
    }

    private static boolean hasFlag(String[] args, String... flags) {
        for (String a : args) {
            for (String f : flags) {
                if (f.equals(a)) {
                    return true;
                }
            }
        }
        return false;
    }

    private static double mean(double[] xs) {
        double s = 0.0;
        for (double x : xs) {
            s += x;
        }
        return s / xs.length;
    }

    /** Bessel-corrected sample standard deviation; returns 0 if there are
fewer than two samples. */
    private static double sampleStdDev(double[] xs) {
        if (xs.length < 2) {
            return 0.0;
        }
        double m = mean(xs);
        double acc = 0.0;
        for (double x : xs) {
            double d = x - m;
            acc += d * d;
        }
        return Math.sqrt(acc / (xs.length - 1));
    }
}

```

```

=== lab5/task3/Task3Main.java ===
package lab5.task3;

```

```

import lab5.common.SimulationConfig;

```

```

import lab5.common.SimulationResult;
import lab5.smo.MultiChannelSmo;

/**
 * Lab 5, part 3: run the SMO while a separate {@code status-reporter}
 * thread periodically prints queue
 * depth, busy servers, and counters ({@link
 * MultiChannelSmo#runWithLiveReporter(long)}).
 */
public final class Task3Main {

    /** Default tick so a ~0.1-0.2 s run still prints at least one status
    line on typical hardware. */
    private static final long DEFAULT_REPORT_MS = 100L;
    private static final long MIN_REPORT_MS = 1L;

    private Task3Main() {}

    public static void main(String[] args) throws Exception {
        long reportMs = DEFAULT_REPORT_MS;
        for (String a : args) {
            if (a.startsWith("-")) {
                continue;
            }
            try {
                long v = Long.parseLong(a);
                reportMs = Math.max(MIN_REPORT_MS, v);
            } catch (NumberFormatException e) {
                System.err.println("Ignoring non-numeric argument: " + a);
            }
        }

        SimulationConfig cfg =
            SimulationConfig.fromMillis(
                4,
                8,
                0.08,
                0.25,
                0.06,
                1000,
                2L,
                42L);

        System.out.println("Task 3: simulation in worker threads; status
        lines from a separate reporter thread.");
        System.out.println("Parameters: channels=" + cfg.channels()
            + ", queueCapacity=" + cfg.queueCapacity()
            + ", minServed=" + cfg.minServed()
            + ", seed=" + cfg.randomSeed()
            + ", reporterIntervalMs=" + reportMs);

        MultiChannelSmo smo = new MultiChannelSmo(cfg);
        long t0 = System.nanoTime();
        SimulationResult r = smo.runWithLiveReporter(reportMs);
        long dt = System.nanoTime() - t0;

        System.out.println("--- final result ---");
        System.out.println(r);
    }
}

```



```
        System.out.printf("Wall-clock run time: %.3f s\n", dt /  
1_000_000_000.0);  
    }  
}
```